

9

Practice – Making Your Home Smart

The second project in this book is a smart home application. Here, we are going to put the wireless communication knowledge we attained in the previous chapters into practice. A basic smart home product usually contains three groups of devices. There are sensor devices such as temperature, light, and motion sensors, actuator devices such as alarms, switches, and dimmers, and usually a gateway that provides access to the home device. This grouping doesn't necessarily apply to all smart home solutions, though. Many smart home products combine sensors and actuators into a single device with a direct IP connection to the local network, which lets its users connect directly to the device itself via a web browser or mobile application. We can find smart thermostats (such as Google Nest Thermostat) or smart doorbells (such as Ring Video Doorbell) on the market as examples of this approach.

However, when the sensors and actuators need to be physically located in different locations of a house and communicate with each other, **wireless personal area networks (WPANs)** come in handy. The IEEE 802.15 series standards define such networks. You can find a good explanation of WPAN standards on Wikipedia here: https://en.wikipedia.org/wiki/IEEE_802.15. In this project, we will develop a light sensor, a switch, and a gateway for our smart home product and set up a BLE mesh network with them.

In this chapter, we will cover the following topics:

- Feature list
- Solution architecture
- Implementation

Technical requirements

You can find the code for this chapter in this book's GitHub repository, here: <https://github.com/PacktPublishing/Internet-of-Things-with-ESP32/tree/main/ch9>.

The external libraries can be found here: <https://github.com/PacktPublishing/Internet-of-Things-with-ESP32/tree/main/common>.

In terms of hardware, you will need the following:

- Three devkits: One for each node in the BLE mesh network
- Three LEDs: One for each node
- A relay: For the switch node
- A TLS2561 light sensor: For the light sensor node

To set up the BLE network and test the project, we will use a mobile application from Nordic Semiconductor ASA called **nRF Mesh**. It is available on both iOS and Android platforms.

Check out the following video to see the code in action: <https://bit.ly/3yxFBae>

Feature list

Our smart home product will have the following features:

- Support BLE mesh networking.
- During provisioning, a node blinks an LED indicator to distinguish itself visually.
- A light sensor will be provided. This broadcasts a message to the network when the light level drops below a certain threshold.
- A switch will be provided. When the switch receives an ON/OFF message or a level change notification from the light sensor, it will turn the connected lamp on/off accordingly.

- A gateway will be provided. This provides a web user interface for controlling the switch's state.

Now that we've covered the feature list, we can propose our solution.

Solution architecture

In this section, we will discuss each node type and show how all the components work together to accomplish the project's goals. Although the feature list described some of the common functionality of the nodes, it would be much clearer to discuss each node separately. Let's start with the light sensor.

Light sensor

The light sensor device will have the following features and functionality:

- It integrates a TLS2561 sensor to measure the ambient light level.
- It reads from TLS2561 every second and compares the reading against a predetermined threshold level.
- When the level goes below or above the threshold, it broadcasts a message to the BLE network, indicating a high/low status of the light reading.
- The user can provision the device in a BLE network by using a mobile app. During provisioning, it blinks an integrated LED indicator.

For this set of features, we can design and implement the following software modules in the light sensor firmware:

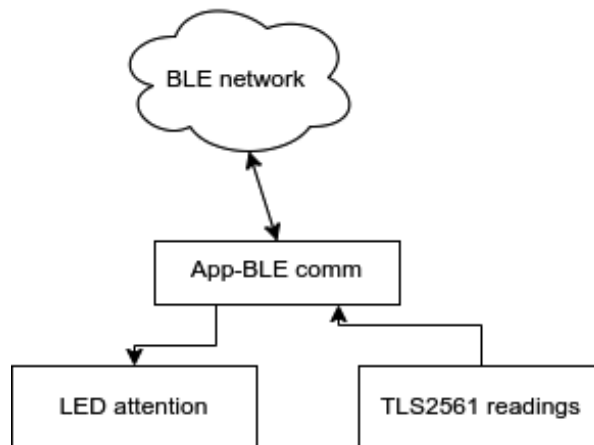


Figure 9.1 – Light sensor firmware modules

The LED attention module abstracts the LED state management. The BLE communication module calls the ON/OFF functions of the LED attention API during provisioning. The TLS2561 module pushes light level changes to the BLE communication module so that they can be broadcasted in the BLE network. The BLE communication module does all the hard work here. It should implement the following BLE mesh models to achieve the intended functionality:

- **Configuration server model:** This model is mandatory for interoperability. A configuration client, typically a mobile application, sends configuration data, including keys, to the device over this model.
- **Health server model:** The main purpose of the health server model is fault reporting and diagnostics. In addition to this, it also helps provision for user attention. We use the LED indicator for this purpose by implementing a callback for the health server events.
- **Generic OnOff server model:** This is the model that delivers a high/low light value in the mesh network.

Any **Generic OnOff client** can receive the light level changes from the light sensor node when the changes are broadcasted in the mesh network via the **Generic OnOff server** model.

The next device we will look at is the switch node, which controls the connected lamp.

Switch

The switch provides the actuator role for the solution. Whenever the light level changes or it is instructed to change by the user, the switch will turn the lamp on/off. The following are its features and functionality:

- The user can provision the device in a BLE network by using a mobile app. During provisioning, it blinks an integrated LED indicator.
- It integrates a relay to control the power of the attached lamp.
- When an ON/OFF message comes from the BLE network, it changes the relay state accordingly. The message source can be the light sensor or a user command.

The switch's firmware will contain the following software modules:

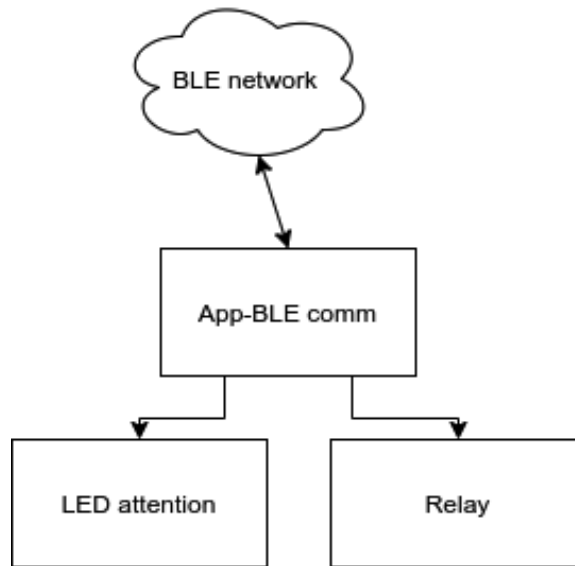


Figure 9.2 – Switch software modules

The BLE communication module has a central role in the firmware. It stands between the BLE network and all the other modules. When a state message comes from the BLE network, it calls the Relay module functions to control the physical relay. The same LED attention module is used in the switch firmware for provisioning.

Let's see which BLE mesh models are included in the implementation of the BLE communication module:

- Configuration server model.
- Health server model.
- **Generic OnOff server model:** This model provides a communication mechanism for changing the state of the integrated relay via other network nodes that implement the Generic OnOff client model.
- **Generic OnOff client model:** This model can connect or listen to the Generic OnOff server models in the network. In our case, the light sensor is the OnOff data source. When a state change notification is published by the light sensor in the network, the Generic OnOff client model will capture it and trigger any related actions.

We will implement both the Generic OnOff server and the client in the switch firmware since the switch serves as the relay controller and consumes the light state changes.

The last node we'll look at is the gateway.

Gateway

One important feature of ESP32 is that it allows us to enable Wi-Fi and Bluetooth simultaneously, which makes it possible to implement a gateway node in our project. This gateway provides a web interface for end users to change the switch state manually via a web browser. The features and functionality of this gateway are as follows:

- It connects to the local Wi-Fi network.
- It runs a web server that publishes a simple HTML form where the user can turn the switch on or off.
- The user can provision the device in a BLE network by using a mobile app. During provisioning, it blinks an integrated LED indicator.

The following diagram shows the modules in the gateway firmware:

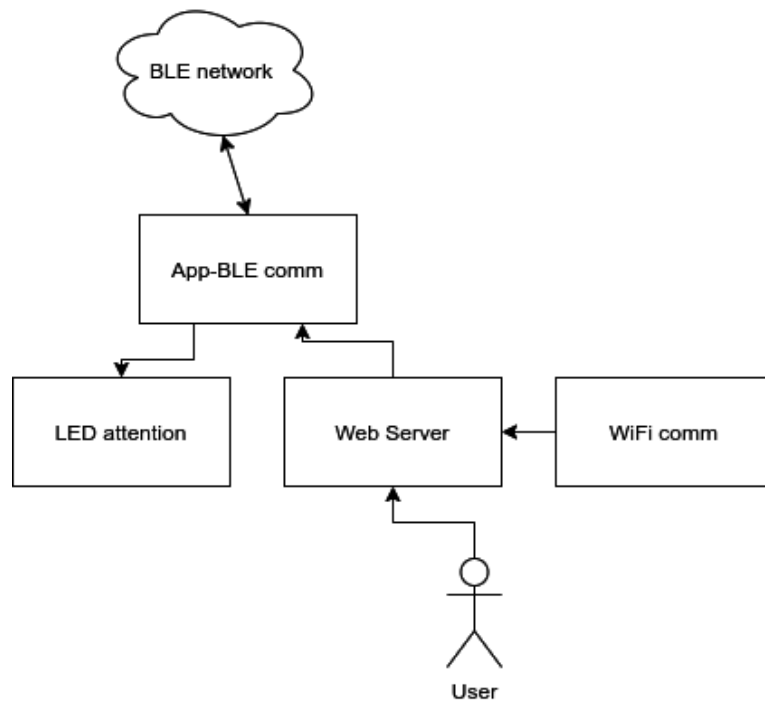


Figure 9.3 – Gateway software modules

In the gateway firmware, we have two additional modules for Wi-Fi communication and the web server. After connecting to the local Wi-Fi, the web server module handles all requests coming from the user and passes switch ON/OFF commands to the BLE communication module.

The following BLE mesh models will be implemented in the firmware:

- Configuration server model.
- Health server model.
- **Generic OnOff client model:** It sends **OnOff** messages to the switch. The **OnOff** parameter comes from the web server.

Before the implementation, let's take a look at the solution:

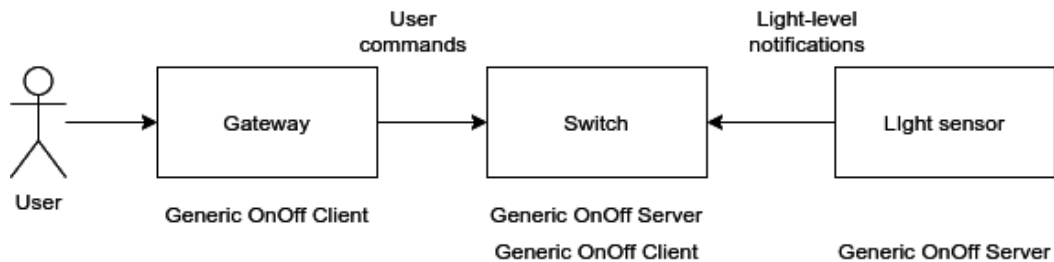


Figure 9.4 – Gateway firmware modules

For the switch, both the gateway and light sensor provide input to change the relay's state. However, the relationship between the gateway and the switch is different from the relationship between the light sensor and the switch. The light sensor, as a server, provides input to the switch in the form of notifications, whereas the gateway sends **OnOff** requests as a client.

In the next section, we will develop the project.

Implementation

Having discussed the solution architecture, next, we will look at the firmware for each device. Let's retain the order we used while discussing the solution and start with the light sensor.

Light sensor

Here, we will integrate a TLS2561 light sensor module into the light sensor device and use an LED as the indicator during provisioning. The following Fritzing diagram shows the connections:

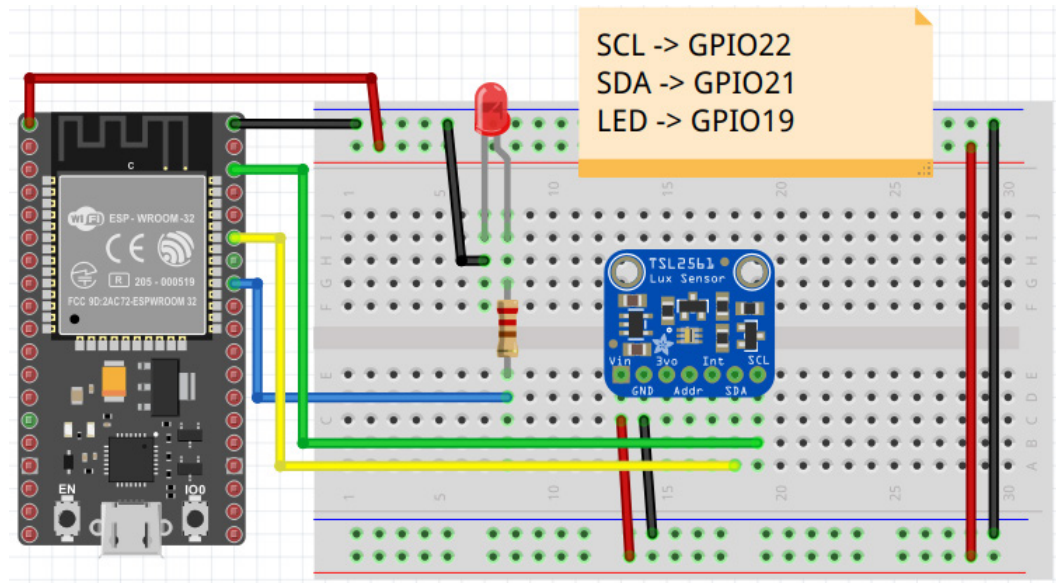


Figure 9.5 – Light sensor diagram

Let's prepare the development environment before coding. Here are the steps:

1. Create a PlatformIO project with the following `platformio.ini` file:

```
[env:az-delivery-devkit-v4]
platform = espressif32
board = az-delivery-devkit-v4
framework = espidf

monitor_speed = 115200
lib_extra_dirs =
    ../../common/esp-idf-lib/components
    ../common
build_flags =
    -DCONFIG_I2CDEV_TIMEOUT=100000
```

`lib_extra_dirs` shows the directories of the external libraries that we will be using. `../common/esp-idf-lib/components` is where the TLS2561 driver resides. The `../common` directory contains the common libraries for all three devices. We will discuss these common libraries later while developing the application.

2. In `sdkconfig`, enable some Bluetooth-related parameters. However, the list is quite long, so we can just copy it from the repository.
3. Activate the virtual environment by using the `pio` command-line tool:

```
$ source ~/.platformio/penv/bin/activate
(penv)$ pio --version
PlatformIO Core, version 5.1.0
```

Now that our development environment is ready, we can start coding. As a spoiler, I want to share the list of source code files so that it will be easier to map them to the solution architecture that we discussed previously:

```
(penv)$ ls src/
app_ble.c  app_ble.h  app_sensor.c  app_sensor.h  CMakeLists.
txt  component.mk  main.c
(penv)$ ls -R ../common/
../common/:
ble  led
../common/ble:
app_blecommon.c  app_blecommon.h
../common/led:
app_ledattn.c  app_ledattn.h
```

The `src/app_ble.{c,h}` files implement the BLE communication module, along with `../common/ble/app_blecommon.{c,h}`. `src/app_sensor.{c,h}` are for the TLS2561 module. Lastly, the LED attention module is implemented in `../common/led/app_ledattn.{c,h}`.

The C source files for the module implementations are a bit lengthy to include here, so we will only talk about the API functions and structures, as declared in the headers. Let's start with `../common/led/app_ledattn.h`:

```
#ifndef appled_attn_h_
#define appled_attn_h_
```

```
#include <stdbool.h>

#define ATTN_LED_PIN 19

void appled_init(void);
void appled_set(bool);

#endif
```

The LED attention API contains only two functions. `appled_init` initializes the GPIO pin of the LED indicator, while `appled_set` updates its state to on or off. The LED attention module is common to all devices.

The next one is `../common/ble/app_blecommon.h`. It declares several BLE functions, all of which are common to all device implementations:

```
#ifndef app_blecommon_h_
#define app_blecommon_h_

#include <stdint.h>
#include "esp_ble_mesh_defs.h"
#include "esp_ble_mesh_config_model_api.h"
#include "esp_ble_mesh_health_model_api.h"

void appble_get_dev_uuid(uint8_t *dev_uuid);
```

First, we must include the `esp_ble_mesh*` header files for the BLE-related structure definitions that we will use in the library functions. `appble_get_dev_uuid` reads the device unique identifier into memory, pointed to by the parameter. The device UUID is needed during provisioning. We have several more functions in this header file, as follows:

```
esp_err_t appble_bt_init(void);

void appble_provisioning_handler(esp_ble_mesh_prov_cb_event_t
event, esp_ble_mesh_prov_cb_param_t *param);

void appble_config_handler(esp_ble_mesh_cfg_server_cb_event_t
event, esp_ble_mesh_cfg_server_cb_param_t *param);
```

```
void appble_health_evt_handler(esp_ble_mesh_health_server_cb_
event_t event, esp_ble_mesh_health_server_cb_param_t *param);
```

`appble_bt_init` initializes the Bluetooth hardware. The others are BLE event handlers, which are common to all device implementations. They handle provisioning, configuration, and health operation events. We also need the attention callback functions, as follows:

```
typedef void (*appble_attn_on_f)(void);
typedef void (*appble_attn_off_f)(void);
typedef struct
{
    appble_attn_on_f attn_on;
    appble_attn_off_f attn_off;
} appble_attn_cbs_t;

void appble_set_attn_cbs(appble_attn_cbs_t);

#endif
```

`appble_set_attn_cbs` sets the attention callbacks, which are called in the health event handler. We can use any type of attention indicator here, such as a buzzer or LED. The health event handler doesn't need to know what it is; just on and off functions are enough for the handler.

Let's move on to the light sensor-specific APIs. The `src/app_ble.h` header file defines the API for the BLE communication module:

```
#ifndef app_ble_h_
#define app_ble_h_

#include <stdlib.h>
#include <stdbool.h>
#include "app_blecommon.h"

void init_ble(appble_attn_cbs_t);
void appble_update_state(bool onoff);

#endif
```

The BLE communication module has a very simple API. It is initialized with `init_ble`, which takes attention callback functions as the parameter. To update the internal state of the Generic OnOff server, `appble_update_state` is called. It also publishes the state change in the BLE network in the form of a notification to be processed by clients. We won't talk about the whole implementation, but it is worth mentioning the models defined in `src/app_ble.c`:

```
static esp_ble_mesh_model_t root_models[] = {  
    ESP_BLE_MESH_MODEL_CFG_SRV(&config_server),  
    ESP_BLE_MESH_MODEL_GEN_ONOFF_SRV(&onoff_pub_0, &onoff_  
server_0),  
    ESP_BLE_MESH_MODEL_HEALTH_SRV(&health_server, &health_  
pub_0),  
};
```

`root_models` is a global array that holds the server models of the light sensor. It contains a configuration server, a health server, and a Generic OnOff server. ESP-IDF provides macros to declare models easily.

Another important point in the source file is that we must register some callback functions as BLE event handlers in `init_ble`:

```
esp_ble_mesh_register_prov_callback(appble_provisioning_  
handler);  
esp_ble_mesh_register_config_server_callback(appble_config_  
handler);  
esp_ble_mesh_register_generic_server_callback(generic_  
server_cb);  
esp_ble_mesh_register_health_server_callback(appble_health_  
evt_handler);
```

The registered callbacks for the BLE events are as follows:

- **Provisioning handler:** This is declared in `app_blecommon.h`.
- **Configuration server handler:** This is declared in `app_blecommon.h`.
- **Generic server handler:** This is implemented in `app_ble.c`.
- **Health server handler:** This is declared in `app_blecommon.h`.

These handlers manage the whole data flow according to the events coming from the ESP-IDF BLE layer.

`src/app_sensor.h` defines the TLS2561 sensor API. Let's see what is inside:

```
#ifndef app_sensor_h_
#define app_sensor_h_

#include "tsl2561.h"

#define LIGHT_THRESHOLD 30
#define LIGHT_SDA 21
#define LIGHT_SCL 22
#define LIGHT_ADDR TSL2561_I2C_ADDR_FLOAT

typedef void (*light_changed_f)(bool);
void init_hw(light_changed_f);
bool is_light_low(void);

#endif
```

The threshold value to report a change is 30 lux. When the light level goes below this value, the library reports it as low-level light, and vice versa. The `init_hw` function initializes the TLS2561 sensor with I²C pins. It also takes a callback function as its parameter, which is to be called when the light level changes.

Now that we've discussed all the libraries, we are ready to develop the application in `main.c`:

```
#include <stdio.h>
#include <string.h>
#include "esp_log.h"

#include "app_sensor.h"
#include "app_ble.h"
#include "app_ledattn.h"
```

First, we must include the header files of the modules. They are as follows:

- `app_ble.h` for the BLE communication module
- `app_sensor.h` for the TLS2561 sensor module
- `app_ledattn.h` for the LED attention module

Then, we define the attention callbacks:

```
#define TAG "sensor"

static void attn_on(void)
{
    appled_set(true);
}

static void attn_off(void)
{
    appled_set(false);
}
```

We call the `appled_set` function with `true` and `false` to change the attention state to on and off respectively. Finally, we define the `app_main` function:

```
void app_main(void)
{
    init_hw(appble_update_state);
    appled_init();
    appble_attn_cbs_t cbs = {attn_on, attn_off};
    init_ble(cbs);
}
```

In `app_main`, we call `init_hw` to initialize TLS2561. The callback function for the light level change is `appble_update_state`, which broadcasts this change in the BLE network. The other initialization function, `init_ble`, is called with the attention callbacks. We connected all the modules at this level by passing the callbacks to each other.

At this point, our program is ready to be compiled and flashed into the devkit. Make sure you have all the source code in place, as I showed at the beginning of this section:

```
(penv)$ pio run -t upload && pio device monitor
```

If everything goes well, we should be able to see the light level printed on the serial terminal every second. The light sensor is ready, so we will develop the switch next.

Switch

For the switch, we will use a relay and an LED. The ultimate purpose is, of course, to control a lamp, but we are good without it in this example.

Tip

If you decide to control a lamp with the relay, you can follow the instructions at <https://ncd.io/relay-logic>. It also explains more interesting connections with relays; for instance, how to connect a three-way light switch so that either relay toggles the lamp. Please make sure you take all the necessary precautions while working with the high voltage.

The hardware connections are as follows:

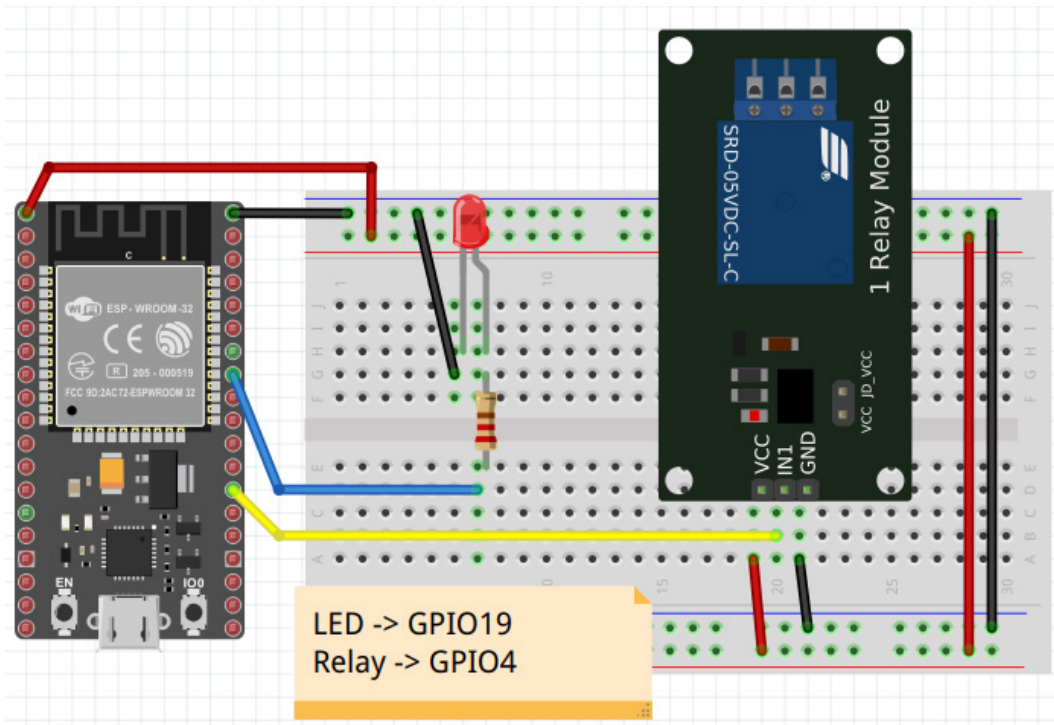


Figure 9.6 – Fritzing diagram for the switch

The steps to prepare the development environment are the same as the ones for the light sensor, so we only need to see `platformio.ini`:

```
[env:az-delivery-devkit-v4]
platform = espressif32
board = az-delivery-devkit-v4
framework = espidf

monitor_speed = 115200
lib_extra_dirs = ../common
```

We have only set the baud rate for the serial monitor and specified the common library folder.

For the source code, we have the following files:

```
(penv)$ ls src/
app_ble.c  app_ble.h  app_sw.c  app_sw.h  CMakeLists.txt
main.c
```

`app_ble.{c,h}` implement the BLE communication module, while `app_sw.{c,h}` implement the relay driving module. Let's see what we have in `app_ble.h` for the BLE communication API:

```
#ifndef app_ble_h_
#define app_ble_h_

#include <stdlib.h>
#include <stdbool.h>
#include "app_blecommon.h"

typedef void (*switch_set_f)(bool);
typedef bool (*switch_get_f)(void);

typedef struct {
    switch_set_f sw_set;
    switch_get_f sw_get;
    appble_attn_on_f attn_on;
    appble_attn_off_f attn_off;
```

```

} app_ble_cb_t;

void init_ble(app_ble_cb_t callbacks);

#endif

```

`init_ble` takes a parameter that contains a list of callback functions. These are as follows:

- `sw_set` and `sw_get`: Switch state get/set functions
- `attn_on` and `attn_off`: Attention indicator ON/OFF functions

The `init_ble` function also registers some functions as the BLE event handlers in its implementation in `app_ble.c`, as shown here:

```

esp_ble_mesh_register_prov_callback(appble_provisioning_
handler);

    esp_ble_mesh_register_config_server_callback(appble_config_
handler);

    esp_ble_mesh_register_health_server_callback(appble_health_
evt_handler);

    esp_ble_mesh_register_generic_server_callback(generic_
server_cb);

    esp_ble_mesh_register_generic_client_callback(generic_
client_cb);

```

`generic_server_cb` is used to handle requests coming from any Generic OnOff client in the network. In our case, it is the gateway, which directs the user commands to the switch as BLE messages. `generic_client_cb` catches the notifications from the light switch. These functions use `sw_set` to change the switch state.

The models of the switch also reside in `app_ble.c` as a global variable, as shown in the following code snippet:

```

static esp_ble_mesh_model_t root_models[] = {
    ESP_BLE_MESH_MODEL_CFG_SRV(&config_server),
    ESP_BLE_MESH_MODEL_GEN_ONOFF_SRV(&onoff_pub_0, &onoff_
server_0),
    ESP_BLE_MESH_MODEL_GEN_ONOFF_CLI(&onoff_cli_pub, &onoff_
client),

```

```
    ESP_BLE_MESH_MODEL_HEALTH_SRV(&health_server, &health_
pub_0),
};
```

Again, we are using the same macros to define the models. This time, we have a new addition to the club: the Generic OnOff client.

Let's see what the Relay API provides in `app_sw.h`:

```
#ifndef app_sw_h_
#define app_sw_h_

#include <stdbool.h>
#define RELAY_PIN 4

void init_hw(void);
void switch_set(bool);
bool switch_get(void);

#endif
```

This is a very clear API with three functions. `init_hw` initializes the GPIO pin that the relay is connected to. We can set the relay state by using `switch_set` and we can call `switch_get` when we need to know its state.

Now that we've looked at all the APIs, we will discuss the main application, which is implemented in `main.c`:

```
#include <stdio.h>
#include <string.h>

#include "esp_log.h"

#include "app_ble.h"
#include "app_sw.h"
#include "app_ledattn.h"
```

We must include the necessary module headers here, which are as follows:

- `app_ble.h` for the BLE communication module
- `app_sw.h` for the Relay module
- `app_ledattn.h` for the LED attention module

Next, we define the attention callback functions:

```
#define TAG "switch"

static void attn_on(void)
{
    appled_set(true);
}

static void attn_off(void)
{
    appled_set(false);
}
```

Now that the attention callbacks are ready, we can continue with `app_main`:

```
void app_main(void)
{
    init_hw();
    appled_init();

    app_ble_cb_t cbs = {
        .sw_set = switch_set,
        .sw_get = switch_get,
        .attn_on = attn_on,
        .attn_off = attn_off,
    };
    init_ble(cbs);
}
```

After initializing the relay and the LED in `app_main`, we must call the `init_ble` function with the callbacks. The most important callback is `sw_set`, which is connected to `switch_set` of the Relay API. It will run any time the BLE communication module receives a notification or a command from the BLE network to change the state of the relay.

Now that the application is complete, we can upload the firmware, as shown here:

```
(penv)$ pio run -t upload
```

That's it. Next, we can develop the last device in the project: the gateway.

Gateway

In terms of setting up the hardware, we don't need anything except the LED connection to the devkit. After attaching an LED to GPIO19, we can start to develop the application. Let's prepare the development environment:

1. Create a PlatformIO project with the following `platformio.ini`:

```
[env:az-delivery-devkit-v4]
platform = espressif32
board = az-delivery-devkit-v4
framework = espidf

monitor_speed = 115200
lib_extra_dirs = ../common
board_build.partitions = partitions.csv
build_flags =
    -DWIFI_SSID=${sysenv.WIFI_SSID}
    -DWIFI_PASS=${sysenv.WIFI_PASS}
```

In `platformio.ini`, we need to provide a custom partitions file because the firmware size will exceed 1 MB, which is the maximum size in the default partitions table. Therefore, we need to increase the size of the app partition to 2 MB in `partitions.csv`.

2. Add the partition file, called `partitions.csv`:

nvs,	data,	nvs,	0x9000,	16k
otadata,	data,	ota,	0xd000,	8k
phy_init,	data,	phy,	0xf000,	4k
factory,	app,	factory,	0x10000,	2M

We have set the size of the app partition to 2 MB here.

3. Copy `sdkconfig` to the root of the project from this project's GitHub repository. It includes all the necessary settings for the custom partitions file, Bluetooth, and BLE.
4. Activate the virtual environment to be able to use the `pio` tool and set the environment variables for the Wi-Fi SSID and password:

```
$ source ~/.platformio/penv/bin/activate
(penv)$ export WIFI_SSID='\ "<your_ssid>" '
(penv)$ export WIFI_PASS='\ "<your_passwd>" '
```

5. Run `menuconfig` and increase the HTTP header length to 4096 for requests. It is located at **(Top) | Component config | HTTP server | Max HTTP Request Header Length**:

```
(penv)$ pio run -t menuconfig
```

Before discussing the code, let's see which source files we have in the project:

```
$ ls src/
app_ble.c  app_ble.h  app_ip.c  app_ip.h  app_web.c  app_web.h
CMakeLists.txt  main.c
```

`app_ble.{c,h}` implement the BLE communication module, while `app_ip.{c,h}` implement the Wi-Fi connection module, as described in the solution architecture. The web server module is implemented in `app_web.{c,h}` and the `app_main` function resides in `main.c` as usual.

Let's start by implementing BLE communication. `app_ble.h` contains the function declarations:

```
#ifndef app_ble_h_
#define app_ble_h_

#include <stdlib.h>
#include <stdbool.h>
```

```
#include "app_blecommon.h"

void init_ble(appble_attn_cbs_t);
void appble_set_switch(bool);

#endif
```

`init_ble` initializes the BLE mesh communication and registers the event handlers. `appble_set_switch` sends Generic OnOff messages to the BLE network. The purpose is to pass the user requests from the IP network to the BLE network. In `app_ble.c`, we also declare the models to achieve the necessary BLE mesh features, as follows:

```
static esp_ble_mesh_model_t root_models[] = {
    ESP_BLE_MESH_MODEL_CFG_SRV(&config_server),
    ESP_BLE_MESH_MODEL_GEN_ONOFF_CLI(&onoff_cli_pub, &onoff_
client),
    ESP_BLE_MESH_MODEL_HEALTH_SRV(&health_server, &health_
pub_0),
};
```

Here, we have the following:

- Configuration server model
- Health server model
- Generic OnOff client model

`app_ip.h` contains the functions and types for connecting to the local Wi-Fi network, as shown here:

```
#ifndef app_ip_h_
#define app_ip_h_

typedef void (*on_connected_f)(void);
typedef void (*on_failed_f)(void);

typedef struct {
    on_connected_f on_connected;
    on_failed_f on_failed;
} connect_wifi_params_t;
```

```
void appip_connect_wifi(connect_wifi_params_t);

#endif
```

`appip_connect_wifi` is the only function that's declared in this header file. It takes a parameter that specifies the callbacks to be called when the Wi-Fi is connected or failed to connect. This lets any client code know about its status.

`app_web.h` is used to manage the web server running on the gateway:

```
#ifndef app_web_h_
#define app_web_h_

#include <stdbool.h>

typedef void (*set_switch_f)(bool);

void appweb_init(set_switch_f);
void appweb_start_server(void);

#endif
```

This provides a simple API for the web server. `appweb_init` takes a parameter of the type `set_switch_f`, which is the function to be called when the user sends an ON/OFF request via the web interface. `appweb_start_server` starts the web server, hence its name.

In `app_web.c`, both HTTP GET and HTTP PUT handlers are implemented for user requests. The web server publishes the following web page:

```
static const char *HTML_FORM = "<html><form action=\"/\"/>  
method=\"post\">  
  <label for=\"switch_state\">Set switch:</label>  
  <select id=\"switch_state\" name=\"switch_state\">  
    <option value=\"ON\">ON</option>  
    <option value=\"OFF\">OFF</option>  
  </select>  
  <input type=\"submit\" value=\"Submit\">  
</form></html>";
```

This is a simple form that shows an **ON/OFF** option group, along with a **Submit** button.

Finally, we will discuss the main application in `main.c`:

```
#include <stdio.h>
#include <string.h>

#include "esp_log.h"
#include "app_ble.h"
#include "app_ip.h"
#include "app_web.h"
#include "app_ledattn.h"
```

First, we must include the `app_*.h` files to use the module implementations and continue with implementing the callbacks, as follows:

```
#define TAG "gateway"

void wifi_connected(void)
{
    ESP_LOGI(TAG, "wifi connected");
    appweb_start_server();
}

void wifi_failed(void)
{
    ESP_LOGE(TAG, "wifi failed");
}
```

When the local Wi-Fi network is connected, we start the web server. We also need the attention callbacks, which we'll implement here:

```
static void attn_on(void)
{
    appled_set(true);
}

static void attn_off(void)
{

```

```

    appled_set(false);
}

```

Now that all the callbacks are ready, we can glue all the components in `app_main` together, as follows:

```

void app_main(void)
{
    appled_init();
    appweb_init(appble_set_switch);

    connect_wifi_params_t p = {
        .on_connected = wifi_connected,
        .on_failed = wifi_failed,
    };
    appip_connect_wifi(p);

    appble_attn_cbs_t cbs = {attn_on, attn_off};
    init_ble(cbs);
}

```

In `app_main`, we initialize all of the modules. We pass the user requests to the BLE network by calling `appweb_init` with `appble_set_switch` as the callback. The HTTP PUT handler inside the web server will call `appble_set_switch` with ON/OFF commands from the user.

Now, the application is ready to be compiled and flashed into the devkit:

```
(penv)$ pio run -t upload && pio device monitor
```

In the serial output, we can see the ON/OFF requests when we connect to the web server via a browser and press the **Submit** button.

With that, we have set up the applications for all our devices. In the next section, we will set up a BLE network with the devices and test them to see how they behave in different scenarios.

Testing

Testing has two stages: setting up the BLE network and testing the scenarios. We will use our mobile phone during these tests. Follow these steps to set up the BLE network and configure the nodes:

1. Power up the devices and run the nRF Mesh application on your mobile phone. When we go to the **Network** tab of the app and tap on the + (*add*) button, we will see all of our devices waiting to be provisioned:

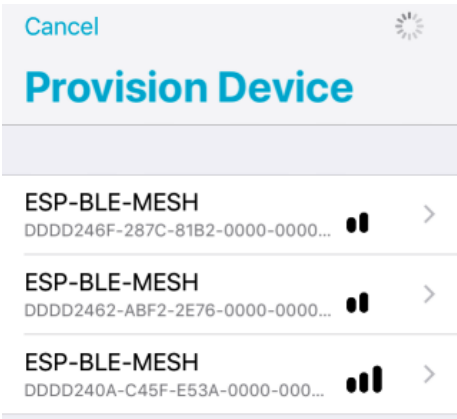


Figure 9.7 – BLE mesh devices ready to be provisioned

2. We can select any of them. When the application connects, the LED indicator will turn on so that we can see which one is connected. In my case, this is the gateway. Rename it gateway and tap **Provision** in the top right:

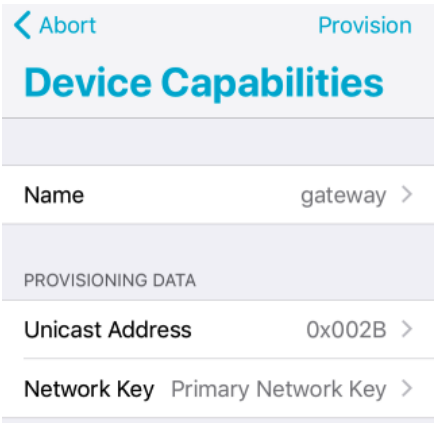


Figure 9.8 – Provisioning

3. After provisioning it successfully, specify an application key to be used by the Generic OnOff server and client models of all devices. It is the shared key of our application:

gateway	
Name	gateway >
Unicast Address	0x002B
Default TTL	7 >
Device Key	DD70C3CDFF71DF43446F2...
Network Keys	1 >
Application Keys	1 >

Figure 9.9 – Specifying the application key

4. Select **Element 1** on the page to see the models that have been implemented in the gateway:

< Node	Element 1
Name	No name >
Unicast Address	0x002B
Location	Unknown
MODELS	
Configuration Server	>
Bluetooth SIG	
Generic OnOff Client	>
Bluetooth SIG	
Health Server	>
Bluetooth SIG	

Figure 9.10 – List of BLE mesh models

5. Bind the application key to **Generic OnOff Client**:

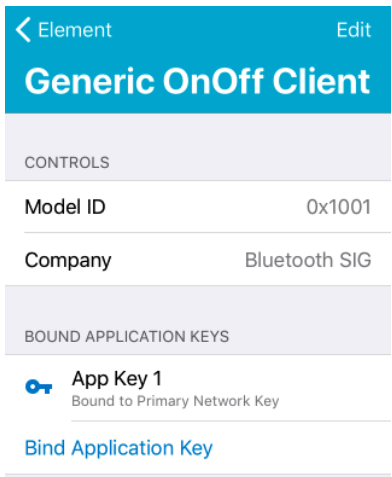


Figure 9.11 – Binding the application key

6. Mark the gateway as **Configured**:

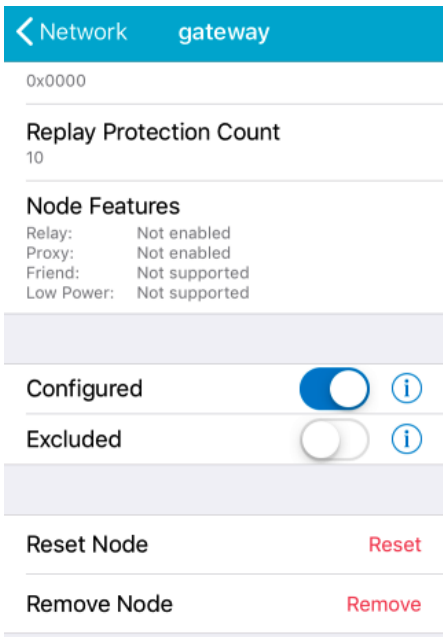


Figure 9.12 – Gateway set to Configured

7. Return to the **Network** view to see the gateway in the **Configured Nodes** list:

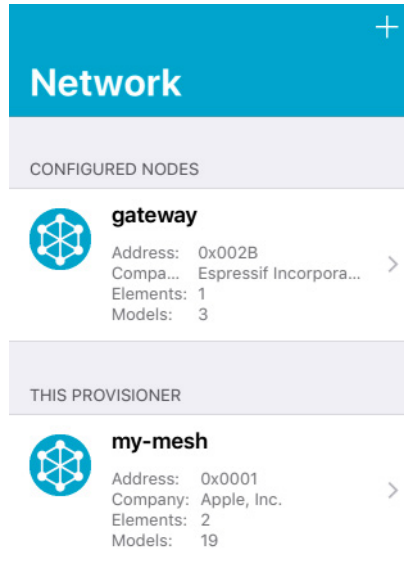


Figure 9.13 – Network view

8. Configure the switch and the light sensor in the same way. Note that there is a slight difference in binding the application key to the models. For the switch, we need to bind the application key to both the Generic OnOff client and the server. For the light sensor, we need to set the publication to **All Nodes** after binding the key to the Generic OnOff server:

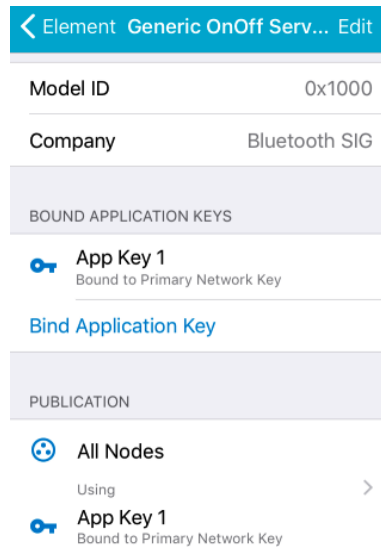


Figure 9.14 – Light sensor publication to All Nodes

Once we have configured all the devices, we can see them as nodes in the BLE network:

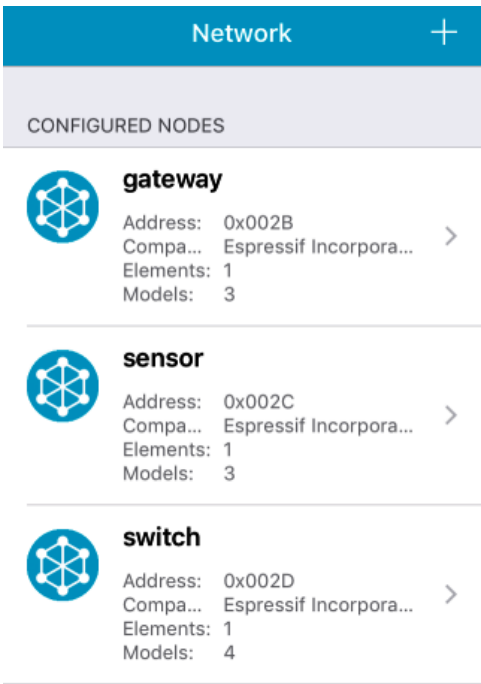


Figure 9.15 – The nodes in the BLE network

Now that the BLE network is ready, we can test the product. There are four basic scenarios that we can test:

1. Sending ON messages from the web interface by connecting to the local IP of the gateway. The switch should set the relay **ON**:



Figure 9.16 – Sending an ON message via the web interface

2. Sending OFF messages from the web interface. The switch should set the relay **OFF**.
3. The light sensor in the dark. The switch should set the relay **ON**.
4. The light sensor in the light. The switch should set the relay **OFF**.

Although we only have three types of nodes in our smart home project, it is open to new types being added to extend the capabilities of the solution. Bluetooth SIG defines more BLE mesh models and if they are not enough, the BLE mesh standard supports vendor-defined models as well.

This project finalizes this section. We are going to learn how to use ESP32 in cloud platforms to harness the real power of IoT in the next chapter.

Summary

With this project, we have completed the part where we explore the wireless connectivity capabilities of ESP32. For IP networks, ESP32 can connect to Wi-Fi. When we need to set up a WPAN to exchange data between devices, it supports BLE mesh networking. As we discussed earlier, IoT cannot be defined without connectivity and communication considerations. In this chapter, we gained important hands-on experience with the networking capabilities of ESP32, which we can leverage in any real-life IoT project.

The next chapter is devoted to cloud platforms and services, and how we utilize ESP32 as an IoT device in cloud solutions to interact with the physical environment remotely. Integration with cloud platforms and services closes the loop for any IoT solution, where all devices can be accessed from anywhere in the world and data coming from those IoT devices is stored for further processing.